

Constitutional Grounding vs. Dynamic Skill Loading: An Empirical Evaluation on Terminal-Bench 2.1

Utkarsh Chandra*
The Supreme Project

Abstract— Agentic systems can place invariant guidance in the initial context or load procedural guidance progressively. We compare three complete configurations—Supreme, Superpowers by obra, and an unprompted baseline—on all 89 Terminal-Bench 2.1 tasks (267 isolated evaluations), using Gemini 3.6 Flash High through Google’s API, with Claude Code as the execution client, at zero temperature. Supreme recorded the highest completion rate (61/89, 68.5%), followed by Superpowers (58/89, 65.2%) and Baseline (55/89, 61.8%). Excluding three upstream environment defects, the corresponding rates were 70.9%, 67.4%, and 64.0%. Supreme also recorded the lowest aggregate wall-clock time (15.15 h vs. 18.30 h and 16.92 h) and the lowest modeled cost per successful task under recorded OpenRouter rate assumptions (\$0.489 vs. \$0.506 and \$0.497). Baseline had the lowest modeled absolute cost (\$27.33; Superpowers \$29.35; Supreme \$29.80), while recorded token totals were 34.52M for Supreme, 33.95M for Superpowers, and 30.87M for Baseline. On Hard tasks, Supreme passed 20/30 compared with 16/30 for Superpowers and 17/30 for Baseline, a directional lead. Paired McNemar tests did not distinguish aggregate completion rates (all $p > 0.05$). Tool telemetry showed lower immediate background-task polling for Supreme, while Task #88 provides a case study of extended exploratory scripting: both Supreme and Superpowers passed, with Superpowers using 11.7 times as many tokens in that run. This evaluation compares complete configurations; it does not isolate guidance delivery from instructional content. The paper presents aggregate results, methods, and analyses; run-level traces and the full evaluation corpus are not part of this release.

1 Introduction

The deployment of frontier large language models as autonomous agents in real-world software engineering and systems administration requires navigating complex, stateful terminal environments [3, 4]. Unlike single-turn question-answering or isolated code generation benchmarks, terminal-grounded tasks demand sustained multi-turn reasoning, file system traversal, compiler diagnostics, and iterative error recovery [8, 11].

As agent systems scale in complexity, a central architectural question is how operational capabilities, procedural guidance, and behavioral constraints should be structured:

1. **Dynamic Progressive Skill Loading:** Exemplified by modular agent frameworks such as Superpowers by obra [10], this design aims to keep initial prompt context compact by exposing a lightweight capability catalog. The agent discovers relevant skills (e.g., test-driven development, systematic debugging, git worktrees) and loads procedural guidance on demand.
2. **Constitutional Static Grounding:** The Supreme configuration draws on constitutional principles [1] and embeds invariant engineering rules (evidence-over-assumption, targeted edits, hierarchical decomposition, and loop management) in its static system-prompt header.

An earlier exploratory evaluation on a simpler 50-task subset (CARB-v2) yielded high pass rates: 94% for Base-

line, 94% for Supreme, and 92% for Superpowers. The reported aggregate comparison was not statistically distinguishable (paired McNemar $p = 1.0$), leaving limited headroom to distinguish outcomes on that subset. We therefore evaluate the complete 89-task Terminal-Bench 2.1 corpus, which covers systems administration, multi-stage compilation, and low-level kernel emulation.

We compare three complete configurations—Supreme, Superpowers by obra, and an unprompted baseline—across the 89 Terminal-Bench 2.1 tasks (267 container evaluations), using Gemini 3.6 Flash High within a shared evaluation harness. The central question is how these configurations compare in task completion, execution economics, and behavior under the same model, harness, and benchmark. This first iteration establishes an empirical baseline for replication and future controlled ablations.

Because Supreme and Superpowers differ in both guidance delivery and instructional content, this comparison does not isolate either factor.

Our investigation yields four primary contributions:

- **Hard-Task Deliberation Patterns:** Supreme’s mean provider-reported deliberation tokens increased by 66.0% from Medium to Hard tasks. On Hard tasks, Supreme passed 20/30 compared with 16/30 for Superpowers (paired McNemar $p = 0.22$), a directional difference that does not establish a causal link between deliberation-token use and task success.
- **Task 88 Case Study:** Both runs passed; Superpowers used 3.51M tokens over 563 turns versus 0.30M over 55 turns for Supreme.

*Project repository: <https://github.com/Utkarsh-X/Supreme>

- Multi-Turn Context Accounting and Cache Telemetry: Provider telemetry recorded 411.3 million cache-read tokens for Superpowers, compared with 322.8 million for Supreme and 295.9 million for Baseline. These observed totals do not isolate how much of the difference is attributable to dynamically loaded documentation.
- Polyglot Domain Breakdown and Esoteric & Symbolic Tasks: Across five engineering domains, Supreme recorded the highest pass rates in Low-Level Systems and SysAdmin & DevOps, while the unprompted Baseline recorded the highest rate in the Esoteric & Symbolic domain (85.7%), which includes COBOL and Scheme tasks.

2 Related Work

Autonomous Software Engineering Benchmarks. Early LLM coding benchmarks such as HumanEval and MBPP evaluated isolated functions using synthetic unit tests. SWE-bench [3] introduced real-world GitHub issues requiring repository navigation and code changes. Terminal-Bench 2.1 [4] complements this focus with interactive, multi-container Linux tasks involving shell use, package compilation, daemon orchestration, and runtime debugging.

Agent Prompt Architectures. The foundational ReAct framework [11] interleaves chain-of-thought reasoning traces with action execution. Subsequent research has explored dynamic capability adaptation, such as Toolformer [6], which trains models to invoke APIs on demand. In open-source agent ecosystems, progressive skill-loading systems have emerged. Superpowers by obra [10] formalizes progressive disclosure, enabling agents to discover and read structured procedural skills from disk, with the aim of keeping initial prompt context compact.

Constitutional AI and Guardrails. Constitutional AI [1] studies alignment through explicit principles that guide model behavior. The Supreme configuration applies engineering rules in its static prompt context—including targeted edits, hypothesis-driven debugging, and verification before completion—with the aim of guiding coding agents toward bounded, verifiable actions.

Prompt Caching and Multi-Turn Context Use. Model APIs offer prompt caching [5, 8], which can reduce the cost and latency of repeated context prefixes. Prefix caching generally depends on stable matching prefixes. Changes to agent context, such as inserted skill documents, large file dumps, or tool-schema updates, may affect cache reuse. This study reports provider-reported cache-read totals across three configurations in multi-turn terminal tasks; it does not isolate the effect of any one context change.

3 Agent System Architectures

All three configurations used Gemini 3.6 Flash High through Google’s API at zero temperature ($\tau = 0.0$), with Claude Code as the agent execution client. A common evaluation harness provided the same tool capabilities for shell execution, file reading, file editing, and process management across configurations; the agent-guidance configuration varied between arms.

3.1 Supreme (v1.0): Constitutional Static Grounding

Supreme v1.0 structures agent behavior through a static, invariant constitutional framework permanently positioned in the system prompt header. It consists of five core operational modules:

- Engineering Constitution: Invariant behavioral axioms prioritizing directly verifiable evidence over model assumption, user intent preservation, and proportional scrutiny scaling with action blast radius.
- Operating Protocol: A decision framework enforcing four sequential execution phases: intent extraction, hierarchical decomposition, surgical code modification, and multi-signal verification.
- Surgical Editing Mandate: Explicit diff constraints requiring agents to justify line changes, use targeted line-replacement tools (`replace_file_content`), and strictly avoid opportunistic or cosmetic file rewrites.
- Anti-Premature-Closure: A mandatory verification forcing function requiring agents to ask: "If this implementation were falsely claiming completion, what plausible omission was overlooked?"
- Loop Management Protocol: Hard limits on consecutive failing commands and strict prohibitions against passive background task polling loops.

Under this architecture, the initial system prompt of length $L_{\text{const}} \approx 7,000$ tokens is embedded statically, keeping the system-prompt prefix invariant across turns and preserving a stable prefix for provider-side cache reuse.

3.2 Superpowers by obra: Dynamic Progressive Skill Loading

Superpowers by obra [10] implements two-tier progressive capability disclosure. The initial system prompt contains a compact index summarizing 14 distinct engineering skills:

1. using-superpowers (Meta-discovery & catalog dispatch)
2. systematic-debugging (Root-cause tracing and defense-in-depth)

3. test-driven-development (Red-Green-Refactor TDD cycle)
4. writing-plans (Hierarchical multi-phase execution planning)
5. executing-plans (Systematic plan fulfillment and checkpointing)
6. verification-before-completion (Pre-flight validation gates)
7. subagent-driven-development (Isolated task delegation)
8. requesting-code-review (Pre-commit review requests)
9. receiving-code-review (Critique integration and refactoring)
10. finishing-a-development-branch (Merge sanitation and branch cleanup)
11. dispatching-parallel-agents (Concurrent sub-agent dispatch)
12. using-git-worktrees (Isolated filesystem branching)
13. brainstorming (Design exploration and architecture ideation)
14. writing-skills (Dynamic procedural skill creation)

When an agent determines that a skill is required, it dynamically invokes file-reading tools to ingest the corresponding SKILL.md and nested reference documents into the conversational history. While reducing the initial prompt to a ≈ 580 -token base skill catalog, this on-demand injection inserts reference text directly into the active dialogue turns.

3.3 Baseline: Unprompted Control

The Baseline configuration provides the raw LLM engine with execution capabilities and tool declarations, but omits all auxiliary system instructions, behavioral protocols, and skill catalogs. It serves as the empirical control to measure the baseline competence of the underlying model.

3.4 Methodological Scope: Structure vs. Content

We emphasize that Supreme and Superpowers differ along two dimensions simultaneously: structural delivery (static invariant prompt header vs. on-demand dynamic disk retrieval) and semantic instruction content (constitutional axioms and surgical diff boundaries vs. procedural engineering recipes such as TDD and worktrees). Consequently, the empirical results presented in this work reflect the overall behavior of the complete configurations.

Controlled ablations (e.g., statically injecting Superpowers documentation, or applying constitutional loop controls to dynamic skill loading) will be valuable in future work to cleanly decouple delivery mechanisms from prompt semantics.

4 Experimental Setup

4.1 Benchmark Corpus: Terminal-Bench 2.1

We evaluate across the complete 89-task corpus of Terminal-Bench 2.1 [4]. To analyze real-world polyglot versatility beyond standard Python scripting, we classify all 89 tasks across five distinct technical engineering stacks:

- Low-Level & Systems Engineering ($N = 20$): C, C++, x86/ARM/MIPS Assembly, Redcode, Rust, and OCaml compiler runtimes.
- SysAdmin, OS & DevOps ($N = 15$): Linux kernel parameters, QEMU hypervisor orchestration, Git object reconstruction, OpenSSH, and Bind9 DNS.
- Software Engineering ($N = 26$): Python applications, Cython extensions, complex data pipelines, and multi-file refactors.
- Esoteric & Symbolic Programming ($N = 14$): Scheme metacircular evaluators, COBOL business logic, LaTeX typesetting, Stan probabilistic modeling, and SPARQL ontology queries.
- Machine Learning & Robotics ($N = 14$): PyTorch distributed parallelism, HuggingFace transformers, FastText embeddings, and MuJoCo XML physics modeling.

4.2 Evaluation Isolation and Anti-Leakage Protocol

To ensure benchmark validity and prevent evaluation leakage during task execution, the evaluation harness enforces strict multi-container isolation:

1. Pristine Container Provisioning: An isolated Docker container is spawned from the official benchmark image with bounded memory and CPU allocations.
2. Evaluation Isolation: During active agent problem-solving, the ground-truth verification suite (/tests) is absent from the container filesystem. The captured run logs record 104 exploratory shell searches attempting to locate verification files (e.g., `find / -name "*test*"`); all recorded attempts returned empty results. This indicates that agents did not access ground-truth tests through these searches, though it does not preclude pre-training data exposure.

3. **Per-Task Timeout Enforcement:** Each task operates under its official per-task timeout T_{limit} specified in its benchmark configuration (task.toml). Across the corpus, timeouts range from 600s to 12,000s with a median of 900s (49 of 89 tasks at the median). Heavy systems and simulation tasks (e.g., install-windows-3.11 and winning-avg-corewars) are allocated 3,600s. Upon self-declaration of completion or reaching T_{limit} , the agent process tree is terminated. Recorded wall-clock includes session teardown: 37 of 267 runs exceeded T_{limit} (36 by at most 39s of teardown); one anomalous baseline session (merge-diff-arc-agi-task, 7,035s vs. a 900s limit) is flagged in the audit ledger.
4. **Post-Execution Injection:** The official verifier is copied into the container via `docker cp tests/ container:/tests` only after agent execution has completed.
5. **Multi-Signal Consensus Verification:** An isolated subshell executes `test.sh`, capturing binary reward status, individual CTRF unit test counts, and raw terminal stdout/stderr logs.
6. **Cryptographic Audit Trail:** A SHA-256 hash is computed over all verifier artifacts and committed to the audit ledger.

4.3 Telemetry and Token Accounting

For runs with successfully captured turn streams, token metrics are extracted from provider API response meta-data on each turn:

- **Input Tokens (T_{in}):** Total prompt tokens presented to the model per turn.
- **Cache Read Tokens (T_{cache}):** Prompt tokens retrieved from provider-side prefix cache (`cached_content_token_count`).
- **Deliberation Tokens (T_{think}):** Provider-reported reasoning tokens generated during internal model deliberation (`thinking_tokens`), captured as aggregate counts rather than decoded chain-of-thought text.
- **Output Tokens (T_{out}):** Generated text and tool-call payload tokens (`candidates_token_count`).

Aggregate recorded token totals are calculated from the captured input and output usage fields. Deliberation tokens are shown separately for analysis and are already included in the output total. The canonical Task #44 (mailman) records for Superpowers and Baseline contain a 350,000-token placeholder but no captured turn-level usage; these placeholders are excluded, and no values are imputed for the missing usage.

5 Empirical Results

Across all 89 tasks (267 isolated container evaluations), Supreme achieved the highest task completion (61/89, 68.5%), the lowest aggregate wall-clock time (15.15 h), and the lowest modeled cost per successful task using the recorded OpenRouter rate schedule (\$0.489, approximately 3.4% below Superpowers at \$0.506). Figure 1 provides an executive visual comparison; Table 1 details the complete scoreboard including infrastructure-adjusted cohorts ($N = 86$, $N = 85$).

5.1 Macro Accuracy and Statistical Significance

Supreme achieved 61 verified wins (68.5%), compared to 58 (65.2%) for Superpowers by obra (+3 wins) and 55 (61.8%) for Baseline (+6 wins).

To test whether aggregate pass rate differences are statistically distinguishable, we conducted paired McNemar’s tests with Edwards’ continuity correction. Comparing Supreme vs. Superpowers yields 52 mutual passes, 22 mutual failures, 9 Supreme-only wins, and 6 Superpowers-only wins ($\chi^2 = 0.267, p = 0.605$). Comparing Supreme vs. Baseline yields 12 Supreme wins vs. 6 Baseline wins ($\chi^2 = 1.389, p = 0.239$). Comparing Superpowers vs. Baseline yields 12 Superpowers-only wins and 9 Baseline-only wins ($\chi^2 = 0.190, p = 0.663$).

Supreme’s observed completion-rate lead is directional; paired testing did not distinguish the aggregate differences at conventional thresholds ($p > 0.05$). The comparison also identifies differences in deliberation scaling, tool use, and context economics.

5.2 Deliberation Scaling Patterns by Task Difficulty

Across the 267 executions, the model generated 5,311,737 provider-reported deliberation tokens (`thinking_tokens`). Table 2 disaggregates pass rates and mean deliberation tokens across official difficulty tiers.

Due to the small Easy strata ($N = 4$), pass rates in that tier carry wide confidence intervals and are reported for completeness; no comparative inference should be drawn from them.

On Hard tasks ($N = 30$), Supreme scaled mean deliberation by +66.0% over Medium tasks (from 17.0k to 28.3k thinking tokens/task; Table 2), achieving a 66.7% pass rate (20/30). In contrast, Superpowers generated 23,196 thinking tokens/task with a 53.3% pass rate (16/30).

To examine this difference statistically on the Hard subset, we constructed the paired 2×2 contingency table for Supreme vs. Superpowers: 15 mutual passes, 9 mutual failures, 5 Supreme-only wins, and 1 Superpowers-only win. A paired McNemar test with continuity correction yields $\chi^2 = 1.500, p = 0.221$ (exact two-tailed binomial $p = 0.219$). Because this 4-task margin does not attain

Table 1: CARB-v4 Master Scoreboard across all 89 Terminal-Bench 2.1 tasks on Gemini 3.6 Flash High. Brackets denote Wilson 95% score confidence intervals. Adjusted scores exclude three tasks with verified upstream container environment defects ($N = 86$); a further conservative cohort excluding all four infrastructure-impaired tasks ($N = 85$) is reported in Section 6.

Configuration	Raw Solved	Raw Pass (%)	Adjusted Pass (%)	Total Time (h)	Avg Time / Task (s)	Recorded Tokens	Tokens / Solved
Supreme	61 / 89	68.5% [58.3%, 77.2%]	70.9% (61/86) [60.6%, 79.5%]	15.15	612.9	34,521,410	565,925
Superpowers by obra	58 / 89	65.2% [54.8%, 74.3%]	67.4% (58/86) [57.0%, 76.4%]	18.30	740.2	33,954,008	585,414
Baseline	55 / 89	61.8% [51.4%, 71.2%]	64.0% (55/86) [53.4%, 73.3%]	16.92	684.3	30,867,863	561,234

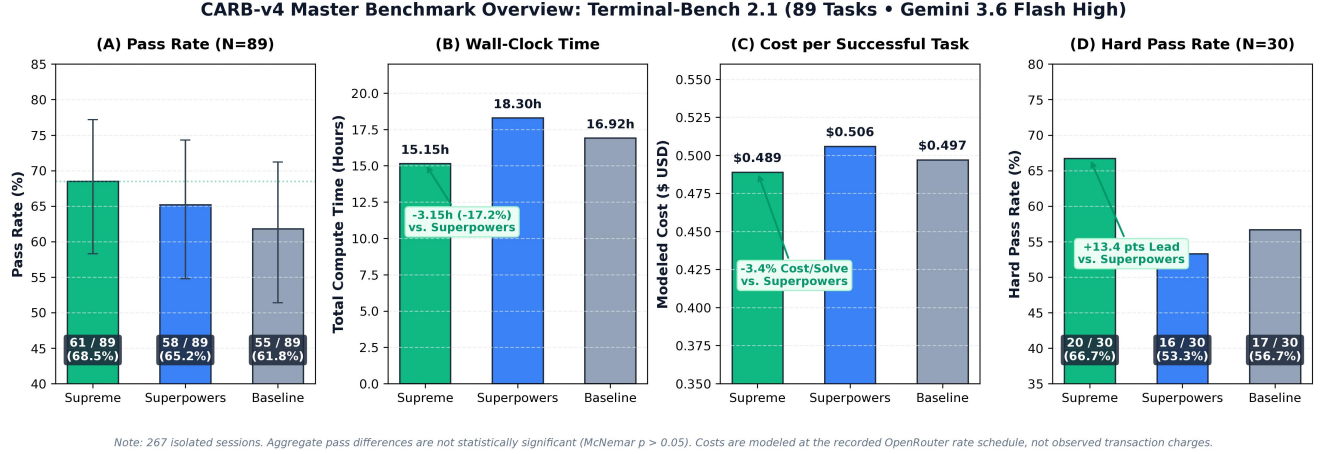


Figure 1: Benchmark outcomes and execution efficiency across all 89 Terminal-Bench 2.1 tasks (267 graded container evaluations). (A) Task completion rates with Wilson 95% confidence intervals. (B) Total wall-clock execution time (lower is better). (C) Modeled cost per successful task using recorded OpenRouter rates (lower is better). (D) Hard-task frontier pass rates ($N = 30$). Supreme achieved the highest task completion (61/89, 68.5%), the lowest aggregate wall-clock time (15.15 h, 17.2% lower than Superpowers), and the lowest modeled cost per successful task (\$0.489, approximately 3.4% below Superpowers). Aggregate accuracy differences are not statistically significant under paired McNemar tests ($p > 0.05$); see Section 5 for full statistical analysis.

Table 2: Deliberation and Win Rates Across Complexity Strata (Wilson 95% CIs in brackets).

Strata	Metric	Supreme	Superpowers	Baseline
Easy ($N = 4$)	Pass Rate	75.0% (3/4) [30.1, 95.4]	25.0% (1/4) [4.6, 69.9]	50.0% (2/4) [15.0, 85.0]
	Mean Deliberation	11,885	14,618	10,688
Medium ($N = 55$)	Pass Rate	69.1% (38/55) [56.0, 79.7]	74.5% (41/55) [61.7, 84.2]	65.5% (36/55) [52.3, 76.6]
	Mean Deliberation	17,035	17,350	18,090
Hard ($N = 30$)	Pass Rate	66.7% (20/30) [48.8, 80.8]	53.3% (16/30) [36.1, 69.8]	56.7% (17/30) [39.2, 72.6]
	Mean Deliberation	28,271	23,196	24,426

significance at $\alpha = 0.05$, we characterize this as a suggestive behavioral pattern (McNemar $p = 0.22$) rather than a confirmed scaling law. In Section 6, we examine transcripts to investigate how dynamic documentation ingestion interacted with reasoning on these tasks.

5.3 Five-Domain Task Success Rates

Table 3 summarizes pass rates across five engineering domains.

Due to sample sizes ($N = 14$ to 26), confidence intervals across stacks overlap substantially. Nonetheless, qualitative trends emerge:

- **Low-Level Systems and SysAdmin:** Supreme demonstrated strong directional pass rates in C, Assembly,

Table 3: Task Success Rate (Pass Rate) Across Five Engineering Domains (Wilson 95% CIs in brackets).

Stack / Domain	Tasks	Supreme	Superpowers	Baseline
1. Low-Level Systems	20	14 (70.0%) [48.1, 85.5]	12 (60.0%) [38.7, 78.1]	13 (65.0%) [43.3, 82.0]
2. SysAdmin & DevOps	15	13 (86.7%) [62.1, 96.3]	11 (73.3%) [48.1, 89.1]	11 (73.3%) [48.1, 89.1]
3. Software Engineering	26	20 (76.9%) [57.9, 88.9]	20 (76.9%) [57.9, 88.9]	15 (57.7%) [38.9, 74.5]
4. Esoteric & Symbolic	14	10 (71.4%) [45.4, 88.3]	9 (64.3%) [38.8, 83.7]	12 (85.7%) [60.1, 96.0]
5. ML & Robotics	14	4 (28.6%) [11.7, 54.6]	6 (42.9%) [21.4, 67.4]	4 (28.6%) [11.7, 54.6]
Total / Overall	89	61 (68.5%) [58.3, 77.2]	58 (65.2%) [54.8, 74.3]	55 (61.8%) [51.4, 71.2]

and OS administration, passing gpt2-codegolf (standalone C inference) and completing the 44-minute install-windows-3.11 installation.

- **Esoteric & Symbolic Tasks:** Unprompted Baseline achieved 85.7% in Esoteric languages (Scheme, COBOL, LaTeX), outperforming both agent systems. Transcript inspection suggests that procedural guidance coincided with additional file-creation activity on these legacy toolchains, consistent with pre-trained model competency being sufficient for these task types without additional scaffolding.

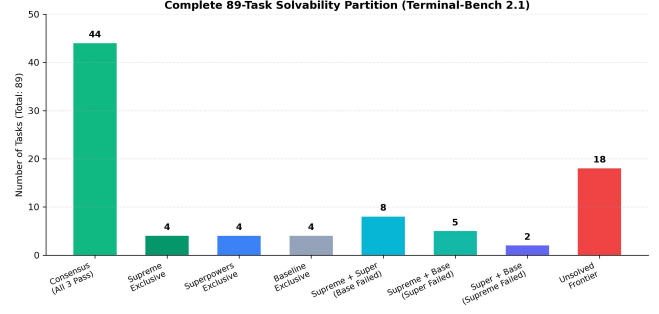
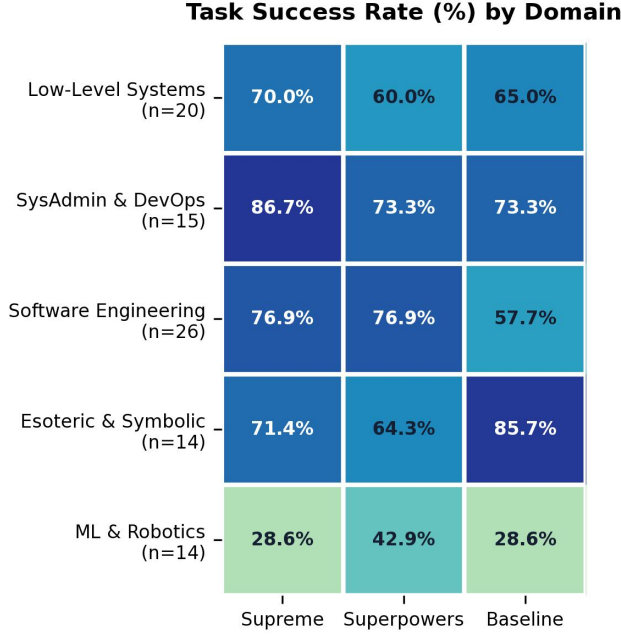


Figure 2: Task success by domain and task solvability. (Left) Task success rate (pass rate, %) for the five technical domains; domain denominators are shown beside each row. (Right) Complete 89-task partition across all eight mutually exclusive outcome subsets.

5.4 The Complete 89-Task Solvability Partition

To examine task overlap without aggregation loss, Figure 2 (Right) categorizes all 89 tasks into eight mutually exclusive subsets ($44 + 4 + 4 + 4 + 8 + 5 + 2 + 18 = 89$ tasks):

1. Consensus Passes ($N = 44$): Solved by all three configurations.
2. Supreme Exclusive Wins ($N = 4$): fix-git, gpt2-codegolf, install-windows-3.11, tune-mjcf.
3. Superpowers Exclusive Wins ($N = 4$): hf-model-inference, mailman, mteb-leaderboard, raman-fitting.
4. Baseline Exclusive Wins ($N = 4$): build-pov-ray, gcode-to-text, qemu-alpine-ssh, regex-chess.
5. Supreme + Superpowers Pass ($N = 8$): Solved by both agent configurations but failed by Baseline (e.g., build-cython-ext, cancel-async-tasks, fix-ocaml-gc, headless-terminal, large-scale-text-editing, merge-diff-arc-agi-task, video-processing, vulnerable-secret).
6. Supreme + Baseline Pass ($N = 5$): Solved by Supreme and Baseline but failed by Superpowers (bn-fit-modify, circuit-fibsqr, overfull-hbox, protein-assembly, rstan-to-pystan).
7. Superpowers + Baseline Pass ($N = 2$): Solved by Superpowers and Baseline but failed by Supreme (largest-eigenval, model-extraction-relu-logits).

8. Unsolved Frontier ($N = 18$): Tasks unsolved by all configurations, including three tasks with verified upstream container bugs.

One Superpowers-only pass (mailman, Task #44) is verifier-graded (3/3 tests). Its stream capture failed; the v4 canonical record therefore has zero turn-level metrics and a 350,000-token placeholder. The pass remains included in completion results and is flagged in the audit ledger. This complete partition establishes that 71 of 89 tasks (79.8%) were solved by at least one configuration, demonstrating complementary problem-solving strengths.

6 Forensic Case Studies and Behavioral Dynamics

To complement aggregate statistics, we examine individual execution transcripts that illuminate the behavioral mechanisms underlying the macro-level differences observed in Section 5. Four tasks across three distinct failure modes illustrate how architectural choices interact with specific problem classes.

6.1 Exploratory Scripting Loops (Task #88: Redcode Simulation)

In Task #88 (winning-avg-corewars), agents were tasked with authoring a Redcode warrior in assembly meeting win thresholds against diverse archetypes in the MARS

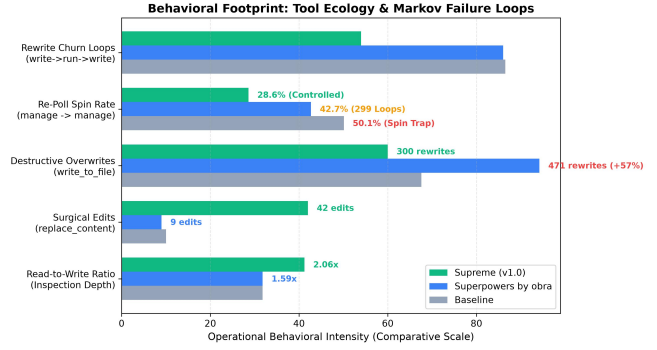
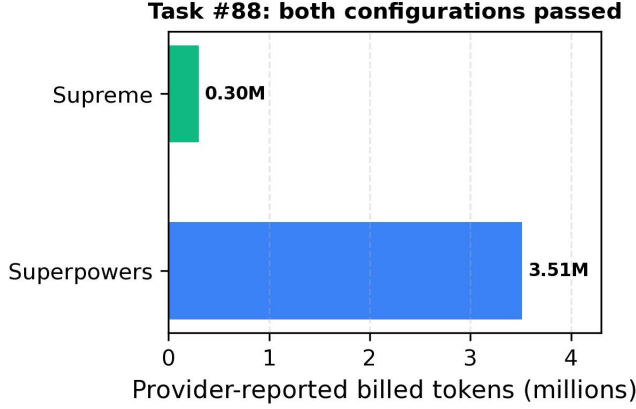


Figure 3: Task-level resource use and tool interaction patterns. (Left) Provider-reported billed tokens for the two successful Task #88 (winning-avg-corewars) runs; bars show measured totals, not per-turn trajectories. (Right) Tool-transition patterns for background-task polling and file-editing strategies.

arena [2]. The task specification defines a 3,600s execution limit ($T_{\text{limit}} = 3600\text{s}$).

The two successful runs differed substantially in measured resource use (Figure 3, Left):

- **Supreme:** Inspected reference warriors, hypothesized that a Silk/Replicator archetype would dominate the arena pool, implemented the candidate warrior, verified win rates, and concluded in 175.0s, 55 turns, and 300,413 tokens.
- **Superpowers by obra:** Entered an extended parameter-search loop and authored 116 Python exploration scripts, repeatedly copying them into the container via docker cp and running battles. The successful run used 563 turns, took 1,648.7s, and consumed 3,513,757 tokens ($11.7\times$ Supreme’s token total for the same successful task).

Task #88 illustrates a tail-risk pattern in open-ended search: exploratory scripting can become prolonged. It is a case study, not a general efficiency ranking. Recorded aggregate token totals remained close, with 34.52M for Supreme and 33.95M for Superpowers.

6.2 Markov Tool Transitions and Polling Dynamics

Analysis of 15,779 logged tool invocations ($P(\text{Next} \mid \text{Current})$ over consecutive transitions) reveals key operational differences (Figure 3, Right):

- **Background Task Polling:** When asynchronous background commands were active, Baseline exhibited a 50.1% probability of immediately re-polling `manage_task`; Superpowers exhibited 42.7% (logging 299 triple-polling sequences (`manage → manage → manage`)). Supreme recorded 28.6%. These are configuration-level observations and do not isolate the effect of any single guidance rule.

- **File Modification Granularity:** Superpowers performed 471 full file overwrites (`write_to_file`) and 172 (`write → run → write`) churn cycles. Supreme utilized targeted line replacements (`replace_file_content`) more frequently (42 vs. 9). While higher usage of targeted editing reflects prompt compliance, it correlated with narrower modification blast radii: Supreme modified a mean of 2.60 unique files per task versus 3.79 on Superpowers.

6.3 Physical Solver Optimization (Task #85: MuJoCo Tuning)

Task #85 (tune-mjcf) required accelerating a MuJoCo physics simulation [9] by $\geq 40\%$ without exceeding trajectory error bounds ($D \leq 10^{-4}$). Supreme achieved an exclusive pass in 414.9s (81 tool calls, 436k tokens), attaining a $2.19\times$ simulation speedup with zero trajectory divergence ($D = 0.0000$). Baseline and Superpowers both failed to reach the speedup threshold ($0.99\times$ and $1.01\times$ speed ratios) due to speculative modifications of solver parameters.

6.4 Long-Horizon Autonomy: Headless QEMU Installation

In `install-windows-3.11` ($T_{\text{limit}} = 3600\text{s}$), agents were tasked with installing Windows 3.11 for Workgroups inside headless QEMU, partitioning virtual disks via `fdisk`, formatting FAT16, swapping floppy images, and installing network drivers. Supreme maintained autonomous multi-step execution across 2,648.7s (44.1 minutes), 307 turns, and 58,010 deliberation tokens to complete the installation. Superpowers and Baseline timed out or corrupted disk partition tables.

6.5 Upstream Benchmark Harness Defects

Forensic inspection identified three distinct upstream environmental failure modes spanning four tasks:

- prove-plus-comm (Task #64): Container metadata error caused immediate execution failure (failed to chdir to cwd ("/app")): no such file or directory across all models.
- caffe-cifar-10 (Task #7): An unreleased package lock on /var/lib/dpkg/lock-frontent held by background PID 168 blocked package installations, causing a starvation timeout across all models.
- Distributed PyTorch Infrastructure (Tasks #82–#83): In Task #82 (torch-pipeline-parallelism), the bare container lacked Python and curl with egress blocked; in Task #83 (torch-tensor-parallelism), a large-scale dynamic PyPI download of 2.85 GB CUDA wheels triggered a verifier timeout before test execution [7].

Excluding the three environmental configuration defects (prove-plus-comm, caffe-cifar-10, and Task #82) yields the $N = 86$ adjusted cohort reported in Table 1 (Supreme: 70.9% [61/86]; Superpowers: 67.4% [58/86]; Baseline: 64.0% [55/86]). Excluding all four infrastructure-impaired tasks yields $N = 85$ (Supreme: 71.8% [61/85]; Superpowers: 68.2% [58/85]; Baseline: 64.7% [55/85]).

7 Discussion and Context Economics

7.1 Context-Growth Dynamics and Multi-Turn Token Economics

As illustrated in Figure 4, multi-turn agent execution exhibits distinctive context characteristics:

1. Input Share of Total Tokens: For Supreme, input tokens comprised 90.5% of recorded total tokens (31,232,870 / 34,521,410). This is an input-to-total ratio; it does not measure what share of input came from prior conversational history. In multi-turn runs, earlier conversation and tool outputs return as later-turn context, so initial prompt size alone does not describe full-trajectory input. Supreme’s invariant header is approximately 7,000 tokens; Superpowers’ initial discovery catalog is approximately 580 tokens and progressively loads procedural documents. These are different architectural layers: the comparison does not support describing Supreme as having the smaller initial context, nor does it quantify the complete skill-corpus sizes.
2. Observed Cache-Read Volume: Provider-reported cache reads totaled 411,332,767 tokens for Superpowers, 322,810,412 for Supreme, and 295,918,105 for Baseline [8]. These are observed cache-read totals; this comparison does not isolate how much of the difference is attributable to dynamically loaded documentation.
3. Terminal Output in Tool Payloads: Terminal stdout (including build logs and compiler output) represented 67.8% of tool-payload tokens in the reported analysis. This points to output volume as a context-management consideration across configurations; it is not a measure of agent efficiency.

Token Use Across the 89-Task Evaluation

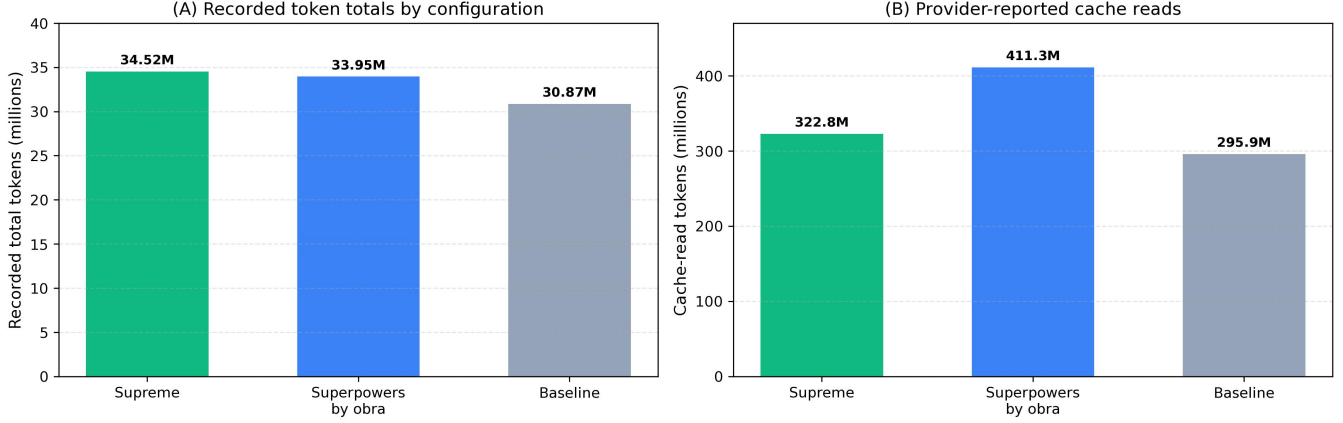


Figure 4: Recorded token usage in the 89-task evaluation. (A) Recorded input-plus-output totals by configuration. (B) Provider-reported cache-read tokens, shown as a separate usage measure; the two panels are not additive components of one total.

7.2 Financial Token Economics and Cost per Solved Task

Table 4 reports modeled API costs across the 267 evaluations under two rate schedules: the Gemini list-price reference (\$0.75 / M input, \$3.75 / M output) and the recorded OpenRouter rates (\$0.5598 / M input, \$3.745 / M output). These are rate-based estimates, not reported transaction charges.

Table 4: Recorded token totals and rate-based cost estimates for the 267-evaluation study.

Economic Dimension	Supreme	Superpowers	Baseline
Input Tokens (T_{in})	31.23M	30.71M	27.71M
Output Tokens (T_{out})	3.29M	3.25M	3.16M
Deliberation Tokens (T_{think})	1.83M	1.71M	1.77M
Recorded Total Tokens (Input + Output)	34.52M	33.95M	30.87M
Cache Read Tokens (T_{cache})	322.8M	411.3M	295.9M
Modeled Total (List-Price Rates)	\$35.76	\$35.21	\$32.62
Modeled Cost / Successful Task (List)	\$0.586	\$0.607	\$0.593
Modeled Total (OpenRouter Rates)	\$29.80	\$29.35	\$27.33
Modeled Cost / Attempted Task (OpenRouter)	\$0.335	\$0.330	\$0.307
Modeled Cost / Successful Task (OpenRouter)	\$0.489	\$0.506	\$0.497
Modeled Hard-Task Cost / Successful Task ($N = 30$)	\$0.680	\$0.730	\$0.645

Note: Component values are rounded independently; deliberation tokens are included in output and shown separately.

Across all 89 tasks, the rate-based cost estimates and outcomes present a three-layer economic portrait. Under the OpenRouter-rate model, Baseline had the lowest modeled total cost (\$27.33), followed by Superpowers (\$29.35) and Supreme (\$29.80). Baseline also had the lowest modeled cost per attempted task (\$0.307), followed by Superpowers (\$0.330) and Supreme (\$0.335). These modeled cost advantages coincide with Baseline’s lower recorded token total (30.87M vs. 34.52M for Supreme), but also with its lower task completion (55/89 vs. 61/89).

Cost per successful task is a composite measure com-

binning modeled total cost with the number of successful tasks; it is not a separately observed charge. On this measure, Supreme had the lowest modeled cost per successful task under the recorded OpenRouter rates (\$0.489), approximately 3.4% below Superpowers (\$0.506), while completing three additional tasks (61 vs. 58). Baseline was \$0.497. The ranking is unchanged under both the list-price and OpenRouter-rate scenarios in Table 4.

On the Hard task subset ($N = 30$), the cost-per-successful-task ranking reverses: Baseline achieved the lowest cost per successful Hard task (\$0.645), followed by Supreme (\$0.680) and Superpowers (\$0.730). Supreme simultaneously achieved the highest Hard-task completion rate (66.7% [20/30] vs. 56.7% [17/30] for Baseline and 53.3% [16/30] for Superpowers), representing a genuine trade-off of superior Hard-task coverage at moderately higher cost per successful Hard solve compared to Baseline.

7.3 Practical Design Implications for Agent Systems

The evaluated configurations suggest three practical design considerations:

Static Invariant Principles: Static placement of invariant engineering principles can avoid mid-turn documentation retrieval while maintaining consistent behavioral constraints across turns.

Loop Termination Heuristics: Explicit iteration thresholds and hypothesis-checking gates help limit prolonged exploratory scripting on open-ended combinatorial problems.

Targeted Line Edits: Surgical diff tools (replace_file_content) correlate with reduced collateral modifications compared to whole-file rewrites.

7.4 Threats to Validity and Limitations

Single Foundation Model: All evaluations used Gemini 3.6 Flash High. Model-specific reasoning and tool policies may interact with prompt architecture; findings may not generalize across model families without multi-model replication.

Single Run per Task: Each task was evaluated with one run per configuration at $\tau = 0.0$ (267 container runs, ≈ 50 compute hours). While $\tau = 0.0$ minimizes sampling variance, multi-seed replication is needed to measure run-to-run stability.

Coupled Structure and Content: Supreme and Superpowers differ in structure (static vs. dynamic) and content (constitutional axioms vs. TDD workflows). Controlled ablations isolating delivery mechanism from content remain future work.

Prompt Freezing: The Supreme constitutional prompt was frozen prior to benchmark execution; no prompt adjustments were made based on benchmark task failures.

8 Conclusion

Across the 89-task Terminal-Bench 2.1 evaluation (267 isolated container runs), the complete Supreme configuration recorded the highest task-completion rate (61/89, 68.5%), the lowest aggregate wall-clock time (15.15 h, 17.2% below Superpowers), and the lowest modeled cost per successful task under recorded OpenRouter rates (\$0.489 vs. \$0.506 for Superpowers and \$0.497 for Baseline). Baseline had the lowest modeled absolute cost (\$27.33) and the lowest recorded token total (30.87M); Supreme’s recorded total was 34.52M and Superpowers’ was 33.95M. Supreme also recorded a directional lead on Hard tasks (20/30 vs. 16/30 for Superpowers and 17/30 for Baseline; paired McNemar $p = 0.22$). Aggregate completion differences were not statistically distinguishable under paired McNemar tests ($p > 0.05$). These results compare complete configurations with different guidance content and delivery mechanisms; they do not establish that static or dynamic delivery alone caused the observed differences. The evaluation provides an empirical baseline for this skill and benchmark; future work should replicate across model families and use controlled ablations to isolate design factors.

References

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. arXiv preprint arXiv:2212.12080, 2022.
- [2] Alexander Keewatin Dewdney. In the game called core war, hostile programs engage in a battle of bits. *Scientific American*, 250(5):14–22, 1984.
- [3] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Mike A. Merrill et al. Terminal-bench: Benchmarking agents on hard, realistic tasks. arXiv preprint, 2026.
- [5] Anthropic Research. Prompt caching: Accelerating context processing in multi-turn agent workflows. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>, 2024.
- [6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053, 2019.
- [8] Google DeepMind Team. Gemini 3.6 flash: High-throughput frontier reasoning and tool use in multi-turn terminal environments. Technical Report, 2026.
- [9] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [10] Jesse Vincent. Superpowers: Dynamic progressive skill extension for autonomous software engineering agents. <https://github.com/obra/superpowers>, 2025.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.